МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2
по курсу «Технологии параллельного программирования»

Обработка изображений на GPU. Фильтры.

Выполнил: Федорова Екатерина Васильевна
Группа: М8О-408Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы: научиться использовать GPU для обработки изображений. Использование текстурной памяти и двухмерной сетки потоков.

Вариант: 4, SSAA. Входные данные: на первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. На следующей строке, два числа wn и hn — размеры нового изображения, гарантируется, что размеры исходного изображения соответственно кратны им. $w*h \leq 4 * 10^8$.

Необходимо реализовать избыточную выборку сглаживания. Исходное изображение представляет собой “экранный буфер”, на выходе должно быть сглаженное изображение, полученное уменьшением исходного.

Программное и аппаратное обеспечение

GPU: NVIDIA GeForce RTX 4050 Laptop GPU
Compute capability: 8.9
Общая память: 6141 MiB, занятая память: 15 MiB, свободная память: 5759 MiB
Графическая память: 5772 MB
Разделяемая память (по умолчанию): 48 KB
Разделяемая память (opt-in, на блок): 99 KB
Разделяемая память (на мультипроцессор): 100 KB
Константная память: 64 KB
Кол-во регистров на блок: 65536
Кол-во регистров на мультипроцессор: 65536
Макс. кол-во потоков на блок: 1024
Макс. размерность блока: 1024 x 1024 x 64
Макс. кол-во потоков на мультипроцессор: 1536
Макс. кол-во блоков на мультипроцессор: 24
Максимальный размер сетки: 2147483647 x 65535 x 65535
Мультипроцессоры: 20

CPU: AMD Ryzen 7 7735HS with Radeon Graphics
Архитектура: x86_64
Address sizes: 48 bits physical, 48 bits virtual
CPU(s): 16
Потоков на ядро: 2
Ядер на сокет: 8
Сокетов: 1
CPU max MHz: 4829,0000
CPU min MHz: 400,0000
Виртуализация: AMD-V
Caches (sum of all):
L1d: 256 KiB (8 instances)
L1i: 256 KiB (8 instances)

L2:	4 MiB (8 instances)
L3:	16 MiB (1 instance)

RAM

	Всего	Занято	Свободно	Общая	Буф/врем.	Доступно
Память	14Gi	7,0Gi	2,3Gi	166Mi	5,9Gi	7,8Gi
Подкачка	2,0Gi	1,2Gi	795Mi			

SSD: DGSM4512GM63T, 476,9G, 0 disk
Файл.система: /dev/nvme0n1p3
Тип: ext4
Размер: 465G
Использовано: 107G

ОС и ПО
Название: Ubuntu 24.04.4 LTS
Версия: 24.04.4 LTS (Noble Numbat)
Ядро: 6.8.0-142-generic
NVIDIA-SMI: 595.84
Driver Version: 595.84
CUDA Version: 13.2
CUDA Toolkit: 12.0 (nvcc V12.0.140)
VSCode: 1.138.0, x64

Метод решения

Для каждого выходного пикселя отдельно усредняются четыре цветовых канала — R, G, B и A. Например, красный канал вычисляется по формуле:

$$R' = \frac{\sum_{y=0}^{k_h-1} \sum_{x=0}^{k_w-1} R_{x,y}}{k_h k_w}.$$

Остальные каналы вычисляются аналогично. Дробная часть результата отбрасывается при преобразовании к целочисленному типу. Полученные пиксели записываются в выходное изображение.

В работе реализованы три версии программы:

1. **CUDA без измерения времени** — выполняет вычисления на графическом процессоре.
2. **CUDA с измерением времени** — выполняет тот же алгоритм и дополнительно измеряет время работы ядра с помощью CUDA Events.
3. **CPU** — выполняет вычисления последовательно на центральном процессоре и служит для сравнения производительности.

В CUDA-версиях используется фиксированная конфигурация запуска. Для обработки изображения произвольного допустимого размера ядро использует циклы с шагом, равным размеру всей сетки по соответствующему измерению (grid-stride loop).

Перед запуском ядра входные данные изображения передаются как 2D текстуры; выходные — как вектор, так как с ними не производятся вычислительные операции и обращения чтения в ядре.

Для оценки производительности измеряется время непосредственно выполнения CUDA-ядра. Время последовательного алгоритма на CPU измеряется средствами библиотеки std::chrono. В обоих случаях измерение проводится отдельно от ввода, вывода и передачи данных между CPU и GPU, что позволяет сравнить длительность самих вычислений.

Описание программы

Состав проекта

Файл	Назначение
lab2_cpu.cpp	эталонная CPU-реализация, использующая последовательный цикл и замер времени через `std::chrono`
lab2.cu	простейшая GPU-версия без тайминга для отправки в чекер
lab2_timer.cu	GPU-версия с таймингом через CUDA events, основная для отчёта
tobin.py	скрипт преобразования картинки в необходимый для лабораторной формат бинарного входного файла
topng.py	скрипт преобразования бинарного файла в картинку

Также в проекте присутствуют тестовые файлы. В папке png сохранены все варианты картинок. В папке data собраны текстовые файлы: fullhd2.txt, fullhd4.txt, fullhd8.txt, 4K2.txt, 4K4.txt, 4K8.txt, 8K2.txt, 8K4.txt, 8K8.txt — входные данные для быстрого тестирования. Также здесь расположены все входные и выходные бинарные файлы .data, однако они игнорируются git’ом.

Основные типы данных

- uint32_t — размеры изображения, координаты и коэффициенты уменьшения.
- uint64_t — накопленные суммы цветовых каналов и вычисление произведения размеров, чтобы избежать переполнения при промежуточных операциях.
- size_t — индексы вектора с компонентами пикселя во избежание переполнения.
- uchar4 — структура CUDA, содержащая четыре 8-битных компонента пикселя RGBA.
- std::vector<unsigned char> — хранение пикселей в оперативной памяти CPU.
- cudaTextureObject_t — объект текстуры, используемый для чтения исходного изображения в CUDA-ядре.

Реализованные ядра

В программе реализовано единственное ядро ssaa. Его сигнатура:

```cpp
__global__ void ssaa(
    cudaTextureObject_t in,
    uchar4* __restrict__ out,
    uint32_t w, uint32_t h, uint32_t kw, uint32_t kh
)
```

Параметры ядра:

- in: входная текстура представления изображения (без байтов с размером, только пиксели);
- out: выходной массив;
- w: ширина выходного изображения;
- h: высота выходного изображения;
- kw: коэффициент сжатия ширины;
- kh: коэффициент сжатия высоты.

Квалификаторы:

- __global__: ядро, вызывается с хоста, исполняется на устройстве;
- __restrict__ для входных указателей: компилятор может агрессивнее оптимизировать (нет пересечений указателей).

Каждый поток получает начальные координаты выходных пикселей на основе своего номера и номера блока. Затем поток обрабатывает пиксели с шагом, соответствующим размеру сетки CUDA.

Для каждого выходного пикселя ядро считывает исходные пиксели из текстуры, суммирует значения четырёх каналов, делит суммы на количество пикселей в блоке и записывает результат в выходной массив.

Исходное изображение копируется в CUDA-массив, доступ к которому осуществляется через текстурный объект. Режим точечной фильтрации cudaFilterModePoint позволяет считывать значения отдельных пикселей. Результат вычислений хранится в видеопамяти, после чего копируется обратно в оперативную память и записывается в файл.

В версии с измерением производительности используются CUDA Events. Измеряется только время выполнения ядра ssaa, без учёта чтения и записи файлов, выделения памяти и копирования данных между CPU и GPU.

CPU-версия

В CPU-версии аналогичные вычисления выполняются непосредственно в функции main() с помощью вложенных циклов. Программа последовательно обходит выходные пиксели, суммирует значения соответствующих исходных пикселей и вычисляет средние значения каналов.

Для измерения времени используется std::chrono::steady_clock. Замер охватывает только вычислительную часть алгоритма, без чтения и записи файлов. Это позволяет сравнить время вычислений CPU и CUDA при одинаковых входных данных и размерах выходного изображения.

Все три версии формируют выходное изображение одинакового формата, что позволяет проверить корректность результатов и сопоставить производительность реализаций.

Вспомогательные функции

Функция checkCuda:
```cpp
bool checkCuda(cudaError_t error, const char* operation);
```
Универсальная проверка кода возврата CUDA. При ошибке печатает имя операции и текст ошибки (cudaGetErrorString) в stderr и возвращает false. Позволяет в main строить цепочки проверок без потери читаемости.

Функции formatError:
```cpp
void formatError(const std::string& context, const std::string& err)
```
Универсальный вывод переданной текстовой ошибки. Печатает в stderr контекст ошибки и её саму в формате «ERROR: ...».

Общая структура main в лабораторных

1. Чтение терминала: inFile, outFile, w, h с проверкой диапазона допустимых значений.
2. Чтение входного файла: ширина W, высота H с проверкой диапазона допустимых значений; затем — массив (байты изображения).
3. (для CPU) Создание cudaArray, текстуры и их настройка; выделение памяти на GPU и копирование данных.
4. Создание событий времени start и stop.
5. Запуск ядра или последовательного алгоритма.
6. (для CPU) Копирование результата и вычисление затраченного времени.
7. Вывод результатов в выходной файл
8. (Для отчётов) Вывод в терминал времени исполнения.
9. Освобождение ресурсов: событий, GPU-памяти, CPU-памяти.

Отладочный вывод и тестирование

Для проверки используются 4 начальные картинки расширением Full HD, 4K и 8K. Каждая из них с помощью скрипта на Python преобразовывались в бинарные файлы, которые использовались для сглаживания с коэффициентами 2, 4 и 8. Для ускорения ввода на вход подавались вспомогательные файлы .txt из папки data. Выходные

бинарные преобразовывались обратно в картинки с помощью второго Python-скрипта. Все изображения расположены в папке png.

Каждый замер производился только единожды.

Результаты

Изображение	Коэф-ты kw, kh	Кол-во блоков	Кол-во потоков (в блоке)	Время GPU [ms]	Время CPU [ms]
1920x1080 (Full HD)	2	1	32	12.6085	29.5311
		1	256	1.8432	
		16	256	0.267136	
		64	1024	0.198656	
		256	1024	0.189568	
		1024	1024	0.263104	
	4	1	32	4.44214	23.0977
		1	256	0.813504	
		16	256	0.26336	
		64	1024	0.239904	
		256	1024	0.183328	
		1024	1024	0.249504	
	8	1	32	3.84525	20.7858
		1	256	1.10902	
		16	256	0.302112	
		64	1024	0.280992	
		256	1024	0.279072	
		1024	1024	0.278656	
3840x2160 (4K)	2	1	32	59.6516	118.19
		1	256	8.05606	
		16	256	0.711744	
		64	1024	0.396768	
		256	1024	0.395936	
		1024	1024	0.40448	
	4	1	32	22.251	91.0653
		1	256	3.24778	
		16	256	0.48304	
		64	1024	0.446848	
		256	1024	0.4424	
		1024	1024	0.443872	
	8	1	32	20.0487	82.5877
		1	256	4.04141	
		16	256	0.55264	

		64	1024	0.500576	
		256	1024	0.468448	
		1024	1024	0.450944	
7680 × 4320 (8K)	2	1	32	238.302	471.79
		1	256	31.5107	
		16	256	2.25542	
		64	1024	1.22262	
		256	1024	1.11533	
		1024	1024	1.10989	
	4	1	32	87.8678	356.313
		1	256	12.2415	
		16	256	1.1657	
		64	1024	1.0961	
		256	1024	1.05715	
		1024	1024	1.03731	
	8	1	32	78.5474	321.888
		1	256	15.3244	
		16	256	1.30074	
		64	1024	1.28422	
		256	1024	1.22515	
		1024	1024	1.2232	

Полученные изображения
1920x1080 (Full HD)



Коэффициент 2 (960x540)



Коэффициент 4 (480x270)



Коэффициент 8 (240x135)



3840x2160 (4K)



Коэффициент 2 (1920x1080)



Коэффициент 4 (960x540)



Коэффициент 8 (480x270)



7680 × 4320 (8K)



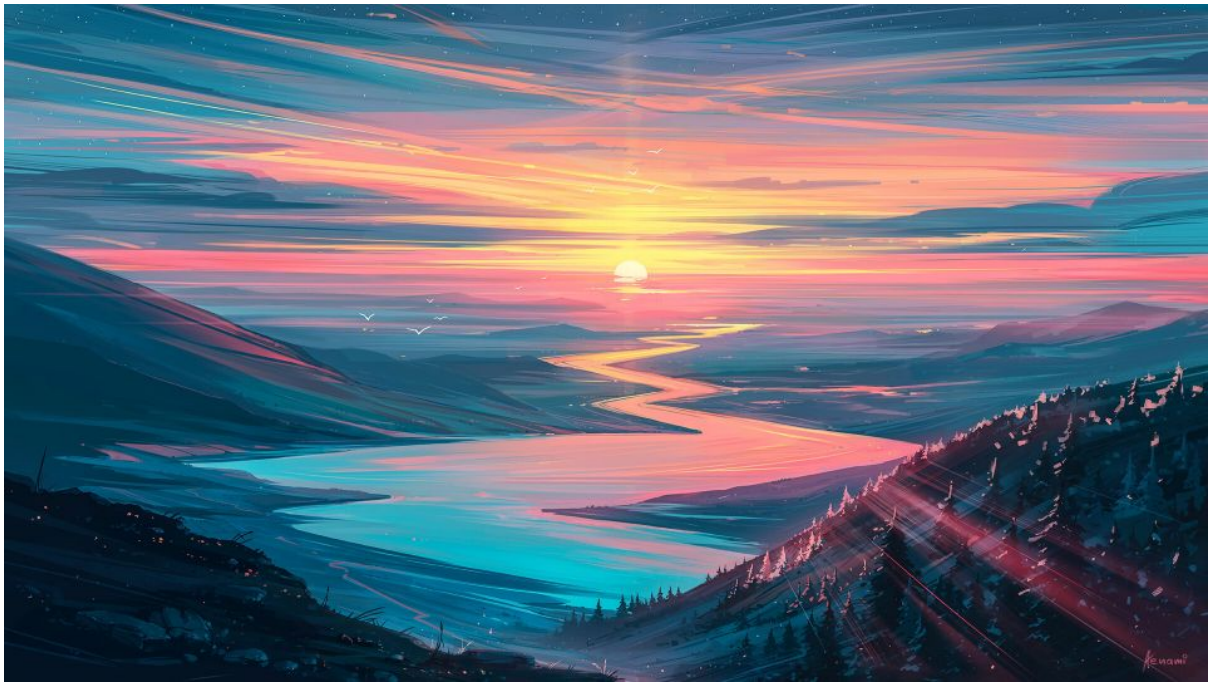
Коэффициент 2 (3840x2160)



Коэффициент 4 (1920x1080)



Коэффициент 8 (960x540)



Выводы

Область применения

SSAA (Supersampling Anti-Aliasing) применяется для уменьшения изображений с одновременным сглаживанием. Для формирования каждого выходного пикселя вычисляется среднее значение цветовых каналов соответствующего блока исходного изображения. Такой подход позволяет уменьшить выраженность ступенчатых границ и сохранить больше визуальной информации, чем простое отбрасывание пикселей. Алгоритм используется в компьютерной графике, обработке изображений и подготовке текстур.

Сложность

Обозначим размеры исходного изображения через $W \times H$, а выходного — через $w \times h$. Коэффициенты уменьшения равны $k_w = W/w$ и $k_h = H/h$.

Временная сложность: $O(WH)$. Каждый исходный пиксель участвует в вычислении ровно одного выходного пикселя, поэтому суммарное количество операций пропорционально числу пикселей исходного изображения. Это справедливо как для последовательной реализации на CPU, так и для CUDA. При этом CUDA распределяет вычисления между потоками и сокращает фактическое время выполнения за счёт параллелизма.

Пространственная сложность: $O(WH+wh) \approx O(WH)$, поскольку необходимо хранить исходное изображение и выходной массив. Для RGBA каждый пиксель занимает 4 байта, поэтому память под изображения составляет приблизительно $4WH + 4wh$ байт. В CUDA-версии дополнительно выделяется память GPU для исходного изображения в CUDA-массиве и выходного массива. Также требуются служебные ресурсы для текстурного объекта и запуска ядра.

Для одного выходного пикселя выполняется $O(k_w k_h)$ операций усреднения. Однако для всего изображения суммарная сложность остаётся $O(WH)$, так как блоки исходных пикселей не пересекаются и полностью покрывают исходное изображение.

Сравнение результатов

Для изображения Full HD (1920 × 1080) время CPU составило от 20,7858 до 29,5311 мс. Лучшее время GPU для коэффициентов уменьшения 2, 4 и 8 составило соответственно 0,189568, 0,183328 и 0,270856 мс. Ускорение относительно CPU составляет приблизительно 156, 126 и 77 раз соответственно.

Для изображения 4K (3840 × 2160) время CPU составило от 82,5877 до 118,19 мс. Лучшие результаты GPU для коэффициентов уменьшения 2, 4 и 8 составило соответственно 0,395936, 0,4424 и 0,450944 мс. Ускорение составляет приблизительно 298, 206 и 183 раза.

На изображении 8K (7680 × 4320) время CPU достигло 471,79 мс при коэффициенте 2, 356,313 мс при коэффициенте 4 и 321,888 мс при коэффициенте 8. Лучшее время GPU

составило 1,10989, 1,03731 и 1,2232 мс соответственно. Ускорение относительно CPU достигает приблизительно 425, 344 и 263 раз.

Во всех рассмотренных случаях CUDA существенно быстрее последовательной реализации на CPU. Наибольшее ускорение наблюдается для изображения 8K при коэффициенте уменьшения 2. При этом увеличение количества блоков и потоков не гарантирует постоянного улучшения результата: после достижения достаточного уровня параллелизма время выполнения меняется незначительно, а иногда увеличивается. Оптимальная конфигурация зависит от размера изображения и особенностей GPU.

Следует учитывать, что измеренное время CUDA отражает только выполнение ядра и не включает передачу данных между CPU и GPU, выделение памяти и операции с файлами. Поэтому приведённое ускорение характеризует именно вычислительную часть алгоритма, а не программу целиком. Кроме того, единичные замеры могут содержать погрешности; для более надёжного сравнения желательно повторять запуски и вычислять среднее или медианное время.

Итог

В ходе лабораторной работы реализован алгоритм уменьшения изображений методом SSAA на CPU и GPU. Эксперименты показали, что CUDA-реализация обеспечивает существенное ускорение, особенно при обработке изображений высокого разрешения. Наибольшее измеренное ускорение составило около 425 раз для изображения 8K.

Также установлено, что изменение количества блоков и потоков влияет на производительность, но не приводит к пропорциональному ускорению. Для выбора оптимальной конфигурации необходимо учитывать размер изображения и проводить повторные измерения. При дальнейшей оптимизации полезно оценить не только время выполнения ядра, но и общие затраты на передачу данных и выполнение всей программы.